TECHNICAL INTERVIEW PREPARATION SERIES

.NET Core, SQL, Architecture & C# Algorithms

Chronological compilation reconstructed from reverse video frames, expanded with industry-standard technical answers and best practices.

Part I: Core .NET Framework & Runtime Architecture

Question 1: What is .NET and How is C# Code Compiled and Executed?

.NET as an Ecosystem: .NET is a cross-platform, modular, and open-source development platform created by Microsoft for building web, cloud, desktop, mobile, and IoT applications. It encompasses the Base Class Library (BCL), language compilers (Roslyn for C#/VB, F#), and the Common Language Runtime (CLR / CoreCLR).

Two-Stage Compilation Pipeline:

1. **Stage 1 (Compile Time):** The Roslyn C# compiler (`csc`) compiles high-level C# source code into **Common Intermediate Language (CIL/IL)**, alongside rich metadata. This output is packaged into portable executable assemblies (`.dll` or `.exe` files). CIL is platform-agnostic CPU instructions.
2. **Stage 2 (Run Time):** When executed, the **Common Language Runtime (CLR)** loads the assembly, performs safety verification, and invokes the **Just-In-Time (JIT) Compiler** (RyuJIT). The JIT compiler converts the IL bytecode into native machine instructions tailored precisely to the host processor architecture (x64, ARM64) on a per-method basis as methods are first called.

> **Modern Optimization (Tiered Compilation & AOT):** Modern .NET uses *Tiered Compilation*: Tier 0 produces quick native code without deep optimizations for fast application startup, while Tier 1 re-JITs frequently executed "hot" methods with loop unrolling and inlining. Furthermore, *Native AOT* allows ahead-of-time compilation directly to machine binaries without requiring a JIT compiler at runtime.

Question 2: Dependency Injection (DI) — Why Does It Matter and How Is It Implemented?

Why DI Matters: Dependency Injection is a design pattern realizing **Inversion of Control (IoC)**. Instead of classes instantiating their own dependencies (e.g., `new OrderRepository()`), dependencies are supplied from the outside by an IoC container. This decouples consumers from concrete implementations, conforms to the Dependency Inversion Principle (SOLID), and enables mock-based unit testing.

Three Types of Dependency Injection:

- **Constructor Injection (Standard & Preferred):** Injected via the constructor and stored in `readonly` fields. Guarantees the object is never in an invalid state and explicitly declares all dependencies.
- **Property / Setter Injection:** Exposed through public properties. Useful for optional dependencies or default fallback implementations, though it can leave objects partially configured.
- **Method / Parameter Injection:** Dependency passed as an argument directly into a specific method invocation (e.g., using ASP.NET Core's `[FromServices]` attribute in action methods). Best when a costly dependency is needed only for a single operation.

Part II: Database Engineering & Query Optimization

Question 3: Database Indexes vs. Stored Procedures vs. Functions

A frequent comparison in database design and performance tuning across enterprise applications:

Concept	Core Purpose & Mechanics	Best Suited Scenarios
Database Indexes	B-Tree or Hash data structures providing logarithmic O(log n) lookup times. Clustered indexes dictate physical row order on disk (one per table); Non-Clustered indexes maintain pointers back to data pages.	Columns heavily referenced in `WHERE` filters, `JOIN` conditions, `ORDER BY`, and `GROUP BY` clauses. (Caution: each index imposes write overhead on `INSERT`, `UPDATE`, and `DELETE`).
Stored Procedures	Precompiled batches of SQL statements executed directly on the database engine. Can return tabular datasets, scalar values, or modify schema/state using transactions (`BEGIN TRAN`). Supports output parameters.	Complex business transactions, multiple mutating operations requiring atomicity, bulk data transformations, and security isolation where callers have execute permissions without table access.
User-Defined Functions (UDF)	Deterministic routines (Scalar or Table-Valued) that compute and return a value or table. *Cannot* perform database schema mutations, write transactions, or use nondeterministic functions like `NEWID()`.	Reusable mathematical calculations, inline string parsing, and data transformations directly within `SELECT` projections and `WHERE` predicates.

Part III: API Security & Memory Management

Question 4: How Do You Implement Security in ASP.NET Core Web APIs?

- **Stateless JWT Bearer Authentication:** Clients authenticate against an identity provider (e.g., Microsoft Entra ID / IdentityServer) and receive a signed JSON Web Token. The API registers `services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)` and validates digital signatures using the authority's JSON Web Key Set (JWKS).
- **Comprehensive Token Validation:** Every incoming request is filtered via middleware validating four critical parameters: `ValidateIssuer`, `ValidateAudience`, `ValidateLifetime` (token expiry with zero clock skew), and `ValidateIssuerSigningKey`.
- **Enforce HTTPS & Security Headers:** Mandate TLS 1.3 via `app.UseHttpsRedirection()` and `app.UseHsts()`. Configure standard security headers (Content Security Policy, X-Frame-Options) and strict CORS policies.
- **Claims-Based & Policy-Based Authorization:** Use `[Authorize(Policy = "AdminOnly")]` with requirement handlers evaluating specific claims, tenant IDs, or scopes (`scp`).
- **Protection Against Vulnerabilities:** Parameterized queries (EF Core) against SQL injection, rate limiting middleware (`app.UseRateLimiter()`) to prevent DDoS/brute force, and Anti-CSRF validation for cookie-based endpoints.

Question 5: Boxing, Unboxing, and String Immutability in C#

Boxing vs. Unboxing:

- **Boxing:** The implicit conversion of a value type (stored on the stack or inline) to a reference type (`object` or an implemented interface). The CLR allocates an object wrapper on the managed heap and copies the value bytes into it.
- **Unboxing:** An explicit conversion from an object reference back to a value type. The CLR checks type compatibility at runtime and copies the value back to the stack.
- **Performance Impact:** High-frequency boxing creates significant Garbage Collector pressure on Gen 0. Using generics (`List<int>` instead of `ArrayList`) eliminates boxing entirely.

String Immutability: In .NET, `System.String` objects are immutable; once allocated on the heap, their characters cannot be modified. Any concatenation operation (such as `str += "abc"`) creates a completely new string object and abandons the old one. For iterative string construction, use **`StringBuilder`** (or `ValueStringBuilder` / interpolated string handlers) to mutate an internal buffer without reallocating.

Question 6: CLR Garbage Collection and the IDisposable Pattern

Generational Garbage Collection: The CLR automatically manages memory allocation on the managed heap using a generational tuning algorithm based on the heuristic that newer objects have shorter lifespans:

- **Generation 0:** Newly allocated small objects. Collected frequently with minimal pause times.
- **Generation 1:** Serves as a buffer between short-lived and long-lived objects.
- **Generation 2:** Long-lived objects (e.g., static variables, caches). Also contains the **Large Object Heap (LOH)** for objects ≥ 85,000 bytes.

Managed vs. Unmanaged Resources: The GC tracks managed heap objects. It has no insight into unmanaged OS resources such as database connections, file handles, native memory pointers, or network sockets. These require deterministic release via the `IDisposable` interface and the C# `using` statement (lowered to a `try...finally` block calling `Dispose()`).

Part IV: State Management & SQL Window Functions

Question 7: ASP.NET MVC State Management — Session vs. TempData vs. ViewBag

Feature	Session State	TempData	ViewBag / ViewData
Lifespan	Multi-request user session (persists until timeout or logout).	Subsequent request only (short-lived; destroyed once read).	Single HTTP request lifecycle only.
Backing Store	Server memory, Distributed Redis, or SQL Server.	Backed by Session or secure encrypted cookie provider.	In-memory controller execution context.
Type Safety	Requires explicit type casting or JSON serialization.	Key-value dictionary; requires type casting upon retrieval.	`ViewBag` is `dynamic` (runtime evaluated); `ViewData` is weakly typed.
Primary Use Case	User auth context, cross-page wizard state, shopping carts.	Flash messages (Success/Error banners) in **POST-Redirect-GET**.	Passing lightweight auxiliary view data (dropdown lists, page titles).

Question 8: SQL Ranking Functions — ROW_NUMBER, RANK, and DENSE_RANK

All three functions evaluate over an ordered window (`OVER (PARTITION BY ... ORDER BY ...)`) but handle ties differently:

- **`ROW_NUMBER()`:** Assigns an absolute, unique sequential integer to each row regardless of duplicate values. Sequence: `1, 2, 3, 4, 5`. *Primary use: API pagination (Skip/Take).*
- **`RANK()`:** Gives duplicate values the identical rank, but skips subsequent numbers to reflect the count of preceding rows. Sequence with ties: `1, 2, 2, 4, 5`. *Primary use: Official sports leaderboards.*
- **`DENSE_RANK()`:** Gives duplicate values the identical rank without skipping any numbers in the sequence. Sequence with ties: `1, 2, 2, 3, 4`. *Primary use: Finding the Nth highest salary in an organization.*

Part V: Advanced C# OOP & Distributed Architecture

Question 9: Explicit Interface Implementation & Collision Handling in C#

The Problem: When a single class implements two interfaces containing identical method signatures:

```
public interface IFileLogger { void Log(); } public interface IDatabaseLogger { void Log(); }
```

The Solution: Explicit Interface Implementation

```
public class Logger : IFileLogger, IDatabaseLogger { // Explicit implementation has NO access modifier (public/private) void IFileLogger.Log() { Console.WriteLine("Logged to local file system."); } void IDatabaseLogger.Log() { Console.WriteLine("Logged to SQL Database."); } }
```

Runtime Invocation Mechanics: Explicit methods cannot be invoked directly through the class instance variable (e.g., `logger.Log()` produces a compile-time error). Callers must cast the instance to the specific interface reference:

```
var logger = new Logger(); ((IFileLogger)logger).Log(); // Outputs: Logged to local file system. ((IDatabaseLogger)logger).Log(); // Outputs: Logged to SQL Database.
```

Dependency Injection Follow-up (The DI Trap): What happens if both interfaces are registered in ASP.NET Core DI:
`services.AddScoped<IFileLogger, Logger>();`
`services.AddScoped<IDatabaseLogger, Logger>();`
Answer: The executed logic depends entirely on which interface is injected into the consuming service's constructor. Injecting `IFileLogger` executes the file routine; injecting `IDatabaseLogger` executes the database routine. If registered as Scoped, two distinct instances of `Logger` will be created within the same HTTP request unless registered using factory delegates or singletons.

Question 10: Microservices Architecture, JWT Invalidation (Logout), and Azure AD Integration

1. Architectural Decoupling: In a microservices ecosystem, independent domains (Orders, Inventory, Auth) interact asynchronously via message brokers (RabbitMQ/Kafka) or synchronously through an API Gateway (YARP, Ocelot). Each microservice maintains its own bounded context and database to avoid distributed coupling.

2. The JWT Stateless Logout Dilemma & Solutions: Because JWTs are cryptographically signed and self-contained, an issued token remains valid across all microservices until its `exp` claim lapses. The server cannot natively "delete" a token stored on a client device. Industry best practices to achieve instantaneous invalidation include:

- **Distributed Blocklist / Revocation Store (Redis):** Upon logout, write the token's unique identifier (`jti` claim) to a fast in-memory Redis cache with a Time-To-Live (TTL) set to the token's remaining lifespan. The API Gateway or authentication middleware checks incoming `jti` values against the Redis blocklist.
- **Short-Lived Access Tokens + Refresh Token Rotation:** Issue access tokens with a strict 5–15 minute lifespan, paired with revocable refresh tokens in a database. On logout, revoke the refresh token family; the user is locked out once the short access window expires.
- **User Security Timestamp Claim:** Store a `TokenVersion` or `PasswordChangedAt` timestamp on the user record. If the token's issued-at (`iat`) precedes this value, reject the token.

3. Azure Active Directory (Microsoft Entra ID) Single Sign-On (SSO): Offloads enterprise authentication to Microsoft. Microservices validate tokens against Microsoft's OpenID Connect discovery endpoints (`/.well-known/openid-configuration`) and JWKS public keys. Single Sign-Out invokes Microsoft's front-channel or back-channel logout endpoints, revoking the user's browser-wide session and triggering OpenID Connect session cleanups.

Part VI: Dependency Injection Lifetime Rules & Matrix

ASP.NET Core's built-in IoC container manages three distinct service lifetimes:

- **Transient (`AddTransient`):** Created fresh every time requested. Best for lightweight, stateless services.
- **Scoped (`AddScoped`):** Instantiated once per HTTP request context. Reused across all components processing that single request (e.g., Entity Framework `DbContext`).
- **Singleton (`AddSingleton`):** Instantiated on first request and remains in memory for the entire application lifetime. Best for caches and thread-safe coordinators.

Service Lifetime Compatibility & Captive Dependencies

A **Captive Dependency** occurs when a service with a longer lifetime holds a reference to a service with a shorter lifetime (e.g., a Singleton injecting a Scoped service). This traps the scoped service, keeping it alive for the application's duration and causing memory leaks, thread-safety violations, and broken DbContext tracking.

Injecting Service	Depends on Singleton	Depends on Scoped	Depends on Transient
Singleton	**Safe**	**FORBIDDEN (Captive Dependency)**	**Caution (Transient becomes Singleton)**
Scoped	**Safe**	**Safe (Within same request)**	**Safe**
Transient	**Safe**	**Safe (Scoped to request)**	**Safe**

Part VII: Frontend Engineering — Angular Full-Stack Questions

Angular Fundamentals & Architecture

1. Architectural Pillars: Angular is a TypeScript-based framework structured around five building blocks:

- **Modules (`@NgModule`) / Standalone Components:** Consolidate related components, pipes, and services into functional boundaries.
- **Components:** The UI controllers with a TypeScript class, HTML template, and CSS styles.
- **Templates:** Declarative HTML enriched with Angular directives and template expressions.
- **Services & Dependency Injection:** Reusable singleton or scoped business logic classes decorated with `@Injectable({ providedIn: 'root' })`.

2. `@Input()` vs. `@Output()`: `@Input()` binds data flowing downward from parent to child component. `@Output()` emits events upward from child to parent using an `EventEmitter<T>` instance.

3. Directives — Structural vs. Attribute: Directives attach custom behavior to DOM elements.

- **Structural Directives:** Alter DOM layout by adding or removing elements (e.g., `*ngIf`, `*ngFor`, or modern `@if` / `@for` block syntax).
- **Attribute Directives:** Change the appearance or behavior of an existing DOM element without altering structure (e.g., `[ngClass]`, `[ngStyle]`).

4. Data Binding Modes: String Interpolation (`{{ value }}`, one-way component-to-view), Property Binding (`[src]="url"`, one-way), Event Binding (`(click)="onSave()"`, one-way view-to-component), and Two-Way Binding (`[(ngModel)]="name"`).

5. `constructor` vs. `ngOnInit`: The `constructor` is standard TypeScript called upon class instantiation; its sole purpose is dependency injection wiring. `ngOnInit` is an Angular lifecycle hook called after Angular initializes all input properties (`@Input`), making it the proper place for API calls, data fetches, and setup logic.

Part VIII: C# Coding Interview Solutions — Optimal Algorithms

Algorithm 1: Count Character Occurrences in a String

Task: Given an input string, count the frequency of each character and return a dictionary of occurrences.

```
using System; using System.Collections.Generic; public class CharCounter { // Time Complexity: O(n) | Space Complexity: O(k) where k is unique characters public static Dictionary<char, int> CountOccurrences(string input) { var result = new Dictionary<char, int>(); if (string.IsNullOrEmpty(input)) return result; foreach (char c in input) { if (result.TryGetValue(c, out int count)) result[c] = count + 1; else result[c] = 1; } return result; } }
```

Algorithm 2: Two Sum — Single-Pass Hash Map Solution

Task: Given an integer array `nums` and an integer `target`, find two distinct indices whose values sum to target. Each input has exactly one valid solution; elements cannot be used twice.

```
using System; using System.Collections.Generic; public class Solution { // Time Complexity: O(n) | Space Complexity: O(n) public int[] TwoSum(int[] nums, int target) { // Key: number value, Value: index where it appeared var map = new Dictionary<int, int>(); for (int i = 0; i < nums.Length; i++) { int complement = target - nums[i]; if (map.TryGetValue(complement, out int complementIndex)) { return new int[] { complementIndex, i }; } // Safely record the current number's index if not already present if (!map.ContainsKey(nums[i])) { map[nums[i]] = i; } } return Array.Empty<int>(); } }
```